

Chap 5

Date

No.

一. 内存结构: 程序运行 = RAM 被分成几块区域

区域	存什么	特点
text segment	程序代码	只读
data segment	已初始化全局变量	程序开始就存在
BSS segment	未初始化全局变量	自动初始化为 0
heap (堆)	动态分配内存	手动管理 (malloc) [不 free 就一直存在]
stack (栈)	函数调用, 局部变量	自动管理 [函数结束就没了]

二. 重点函数总结

函数	语法	parameters
malloc	<code>void * malloc(size_t size)</code>	size: 要分配的字节数
<code>#include <stdlib.h></code> calloc	<code>void * calloc(size_t n, size_t size)</code>	n: 元素个数, size: 每一个元素字节数
realloc	<code>void * realloc(void * ptr, size_t new-size)</code>	ptr: 原内存地址, new-size: 新字节数
free	<code>void free(void * ptr)</code>	ptr: 要释放的地址
作用	初始化	返回值 <i>including the existing</i>
分配一块连续内存	X 不初始化 (垃圾值)	{ 成功: 申请的内存起始地址 失败: NULL
分配几个连续元素空间	✓ 全部初始化为 0	
改变内存大小	保留原数据	成功: 新内存空间的地址 失败: NULL
释放内存	—	无

关键注意

realloc: 扩容后, 原有数据会尽量保留

✧ 若无连续空间 } 找一块更大的
若成功, 原内存会被 free } 把旧数据复制过去
(且换 address) } 返回新的地址

`void * p = realloc(arr, ...);` 用临时 pointer

防返回 NULL, 原内存泄漏

失败不会释放原内存, 需 free

free: 传给 free 的, 必须是之前申请的空间的地址

eg: `p = malloc(100);`

`} free(p); ✓`
`free(ptr); X`

三. 课程例子

1. Suppose u r told to write a C program to perform a set of statistical analysis on the fly

```
if ((num = (int *) malloc(sizeof(int))) == NULL) {
    printf("malloc fails\n");
    exit(1);
}
```

A'ZONE

成功读入几个变量

返回n

遇到不匹配

返回已成功读入的个数 (eg. 0)

input 结束

返回EOF (通常是-1)

补充scanf返回值

```
j=0;
while (scanf("%d%c", &num[j]) == 1)
{
    sum += num[j];
    j++;
}
```

/enlarge array/

```
if ((num = (int*) realloc(num, size * (j+1))) == NULL)
```

```
printf("realloc fails.\n");
exit(1);
```

更快:

```
scanf("%c", &this1);
```

free(first); // 防内存泄漏

```
while (this1 != '\n')
```

exit(1);

恢复current位置

count++; // 记录个数好恢复current

```
*current = this1;
```

```
first = temp;
```

```
current = first + count;
```

```
if (count % increment == 0) // 是否需扩容
```

```
buffer size += increments;
```

```
else current++;
```

用临时指针 ← 接住

```
char *temp = realloc(first, buffer size);
```

```
scanf("%c", &this1);
```

```
if (temp == NULL)
```

```
printf("realloc fails.\n");
```

2. Sort in ascending matrix no. selection sort

```
struct student {
```

```
    int st[MAXSIZE];
```

```
int GetRecord()
```

```
{
```

```
    int count = 0, i;
```

```
    FILE *indata;
```

```
    indata = fopen("student.in", "r");
```

用指针数组 do { 别忘了 casting

```
    st[count] = (struct student *) malloc(sizeof(...));
```

开内存, 再用 "→" fscanf(indata, "%s%s%c%c%s%s",

去存 indata 里的数据 st[count] → matrix, ... st[count] → subject[i]);

```
    count++;
```

```
    while (!feof(indata)) // return 1 if EOF has been reached
```

```
    fclose(indata);
```

0 otherwise

```
    return count;
```

```
}
```

```
void SortRecord(int n)
```

```
    int i, j, min;
```

```
    struct student *temp;
```

```
    for (i = 0; i < n; i++)
```

```
        min = i;
```

```
        for (j = i + 1; j < n; j++) if (strcmp(st[j] → matrix, st[min] → matrix) < 0) min = j;
```

```
        if (min != i)
```

```
            temp = st[min];
```

```
            st[min] = st[i]; 直接交换指针
```

```
            st[i] = temp;
```

```
    }
```

```
}
```

3. 已知大小开动态数组

```
int *arr = (int *) malloc(n * sizeof(int));
```